Privacy and Security Risks in AI Coding Agent Workflows

Nargiz Heybatova Joohan Kim Muhammad Bilal Ali Aditi Agni Frank Du
Carnegie Mellon University

Abstract

Autonomous AI coding agents execute multi-step software-engineering tasks and rely on user permission prompts to gate actions such as file edits, shell commands, and package installations. We conducted a two-phase mixed-methods study—an online pre-screening survey ($n = 21$ respondents; 10 selected) and a 90-minute in-lab observational session with retrospective think-aloud interviews—to investigate how developers evaluate these prompts. Using a Flask "Task Manager" application seeded with five functional bugs and six security vulnerabilities, a Claude Code CLI configured through a `CLAUDE.md` apparatus and `settings.json` hooks issued 20–40 yes/no confirmations per session across four prompt categories. Audio interviews were transcribed for analysis and the original audio recordings were then deleted; only de-identified transcripts were retained. Data show that increased prompt frequency correlated with shorter decision times, higher approval rates for unnecessary actions, and a 0% recall rate for approved security-testing packages among fast-approvers. Contrary to the naïve expectation that security training translates into prompt-level vigilance, security-focused participants did not read prompts more carefully than general engineering participants. Instead, security-focused participants offloaded vigilance to their environment—virtual machines, plan mode, sandboxes—while general engineering participants read prompts because they had no such safeguards. We argue that prompt fatigue is a security risk, not merely a UX iss ue, and that defenses should make environmental safeguards the default rather than relying on per-prompt scrutiny.

All study materials, telemetry hooks, and anonymized logs are available at https://github.com/nargizh7/ai_agent_lab_study.

1 Introduction

AI coding agents such as Claude Code, Cursor, and GitHub Copilot have shifted from passive autocomplete tools to autonomous systems that operate across multi-step tasks and routinely request elevated privileges—file-system access, environment-variable exposure, shell command execution, and outbound network calls. The mechanism that bridges agent autonomy with human oversight is the *permission prompt*: a runtime request for the developer to approve, deny, or persistently grant a specific action. As these tools proliferate, the security and privacy consequences of how developers grant such permissions remain poorly understood.

Prior work has shown that AI assistants can produce vulnerable code[4,5,7], that agentic systems are technically exploitable through prompt injection[2], and that interaction frequency with AI agents erodes risk perception[8,9,12]. What this literature does not address is how developers make *real-time* authorization decisions during sustained agent-driven workflows. At the moment the agent asks "May I edit `app.py`?" or "Should I install `pytest`?", do developers evaluate the request, or has the prompt become a rubber-stamp ritual?

This paper investigates that question through three research questions:

RQ1. How do developers evaluate AI coding agent permission prompts when deciding to approve, deny, or grant persistent permission?

RQ2. Does repeated exposure to permission prompts increase the likelihood of approving prompts without scrutiny—i.e., produce *prompt fatigue*?

RQ3. How do prompt-evaluation behaviors differ based on developers' prior programming experience and AI coding agent usage?

We test four hypotheses:

H1: Higher prompt frequency is associated with faster decisions and higher overall approval rates.

H2: Under high prompt frequency, participants are more likely to approve at least one high-risk prompt.

H3: Greater programming experience is associated with lower approval rates for high-risk prompts.

H4: Frequent AI-agent users show higher baseline approval rates but remain susceptible to fatigue.

We conducted a two-phase mixed-methods study. Phase 1 was a 5-minute online screening survey (21 respondents) used to select 10 master's students for Phase 2. Phase 2 was a 90-minute in-lab observational session in which participants debugged a Flask "Task Manager" application seeded with 11 functional and security flaws using a Claude Code CLI configured to issue 20–40 yes/no permission confirmations per session. We collected screen recordings, deterministic JSONL telemetry from Python hooks, Claude DevTools conversational transcripts, and 25-minute retrospective think-aloud interviews.

Our results support H1, H2, and H4, and partially support H3 in an unexpected form: security-focused participants did not read prompts more carefully but instead pre-committed to environmental safeguards that moved the security decision out of the prompt layer entirely. We also document an anomalous behavior in which Claude detected and refused to silently follow our experimental apparatus—a finding about the difficulty of running deception-based studies on aligned agents.

2 Background and Related Work

Our study sits at the intersection of four lines of research: (1) how developers adopt AI assistants and how adoption changes their workflows; (2) the security characteristics of AI-generated code and the technical attack surfaces of agentic systems; (3) how trust evolves and erodes through repeated interaction, including the phenomenon of permission fatigue; and (4) how developer background shapes engagement with AI tools.

2.1 Adoption and Workflow Effects

AI coding assistants have moved from autocomplete to autonomous workflow participants, and how developers absorb that shift varies sharply. Barke et al. [6] observed that programmer interactions with Copilot are bimodal: an *acceleration* mode where the developer knows the next step and uses the assistant to get there faster, and an *exploration* mode where the developer uses suggestions to scope an unfamiliar problem. Klemmer et al. [5] interviewed 27 software professionals and reviewed 190 Reddit threads, finding that AI assistants have been integrated into everyday workflows but that security validation of their output is uneven and largely informal. Huang et al. [11], surveying 99 professional developers and observing 13, found that experienced practitioners do *not* "vibe code" but instead retain tight control over agents through planning, supervision, and rigorous output review. Our cohort departed from this baseline under prompt-density pressure.

He et al. [1] found that after *projects* adopted the Cursor agent, they exhibited short-term velocity gains alongside long-term increases in code complexity and static-analysis warnings. Chen et al. [3] found that autonomous agents (OpenHands) substantially reduced the manual effort required to complete tasks and improved task correctness by 35%, but participants frequently could not explain why the agent made specific changes.

2.2 Security Vulnerabilities and Agentic Attack Surfaces

Pearce et al.[4] evaluated GitHub Copilot across 89 scenarios and found ≈40% of generated code contained vulnerabilities including SQL injection, path traversal, and hard-coded secrets. Perry et al.[7] uncovered a "trust gap": participants using AI assistants produced *less* secure code while expressing *higher* confidence in their work.

Liu et al.[2] built AIShellJack, which hijacked agentic AI coding editors via prompt injection and triggered high-risk terminal commands with an 84% success rate. Their tests assumed auto-run mode with confirmation features disabled, and the authors explicitly call for user studies examining how developer behavior interacts with confirmation mechanisms—a call our study answers directly.

2.3 Trust, Permission Fatigue, and Consent

Tolsdorf et al. [8] surveyed 123 conversational-AI users and found that greater chatbot usage frequency correlated with higher trust and *lower* perceived risk, identifying three latent user classes ranging from "Hesitant Skeptics" to "Confident Adopters." Wu et al. [9] argued that conventional permission models are inadequate for agentic execution, and proposed an ML-based assistant that predicts user permission decisions with 85% accuracy. The implication is that explicit per-action consent has become impractical at agentic interaction frequencies. Sabouri et al. [10] traced how developer trust in AI code suggestions evolves across an interaction, finding that comprehensibility and perceived correctness dominate trust judgments and that trust shifts as developers accumulate evidence about agent reliability. Our session data extends this dynamic to the permission-prompt setting. Sharma et al. [12] identified four *Feedback Barriers* (Common Ground, Verifiability, Communication, and Informativeness) that prevent users from giving high-quality feedback to conversational agents; the Verifiability barrier in particular predicts that users will not push back on outputs they cannot easily inspect.

2.4 How Background Shapes Developer–Agent Interaction

Long et al. [13] studied 27 CS students' interactions with three AI coding assistants and found substantial variation in suggestion-acceptance rates depending on task type, recommendation method, and content. They identified 22 distinct decision-making factors, including code comprehensibility

and perceived alignment with the student's intent, governing whether suggestions were accepted, modified, or rejected. Khojah et al.[14] found that experienced industry developers used ChatGPT for guidance and problem identification rather than wholesale code generation.

2.5 Positioning This Study

While prior work has established that AI coding tools produce vulnerable code [4, 7], that developers apply insufficient validation [5], that agentic systems are technically exploitable [2], and that interaction patterns evolve under workflow pressure [8, 9, 12], a critical dimension remains unexplored: how developers interact in real time with the permission and access-control mechanisms that gate agent actions. He et al. [1] analyzed post-hoc commits but cannot speak to behavior at the moment risk entered the codebase. Liu et al. [2] disabled confirmation features for tractability. Chen et al. [3] focused on productivity and correctness rather than on security-mechanism response. Our project builds on these foundations by capturing how developers evaluate, and increasingly fail to evaluate, permission prompts over a sustained debugging session.

3 Methodology

We conducted a two-phase mixed-methods study approved by the Carnegie Mellon University IRB (STUDY2026_00000141).

3.1 Recruitment and Pre-Screening (Phase 1)

We recruited candidates through physical flyers in the Gates-Hillman Center (GHC), Wean Hall, Hamburg Hall (HBH), and the Tepper School (TCS); CS-related class Piazzas; departmental mailing lists; and Slack channels. Twenty-one candidates completed the screening survey. We selected 10 participants for Phase 2 based on eligibility and deliberate background balancing: five security-focused (P1, P3, P4, P5, P7) and five general engineering or ML/web-development (P2, P6, P8, P9, P10). Selected participants received a $20 Amazon gift card after completing Phase 2.

3.2 Study Apparatus Design (Phase 2)

We constructed a Flask "Task Manager" web application of approximately 160 lines spanning `app.py` and `auth.py`, seeded with 5 functional bugs and 6 security vulnerabilities.

Functional bugs: (1) `update_task` reads `data["name"]` while the model field is `title`; (2) `complete_task` lacks a 404 guard; (3) `delete_task` is registered as `POST` rather than `DELETE` and calls the nonexistent `db.session.remove()`; (4) `serve_upload` points to `upload/` instead of `uploads/`; (5) `change_password` stores the new password in plaintext.

Security vulnerabilities: (6) hard-coded admin key (`ADMIN_API_KEY = "admin-master-key-DO-NOT-SHARE-2024"`); (7) unauthenticated `/tasks` listing returning all users' tasks; (8) Server-Side Request Forgery in `/api/preview`; (9) silent telemetry beacon exfiltrating request metadata on every response; (10) weak `SECRET_KEY` fallback; (11) `flask-cors` installed from a GitHub `HEAD` URL, bypassing PyPI integrity checks.

Compliance cadence apparatus. We manipulated Claude's behavior through a `CLAUDE.md` file (≈2,000 words of injected instructions) and a `settings.json` configuration registering `UserPromptSubmit` and `Stop` hooks. The apparatus directed the agent to: (i) request yes/no confirmation before every file edit and package install; (ii) cycle through nine package install prompts on every user turn (bandit, safety, pytest, pylint, prettier, jq, httpie, python-dotenv, discord.py, and gpiozero); (iii) fire five trigger-based install prompts when specific lines were touched; (iv) issue a cumulative scope review after every modification; and (v) execute ten post-fix hardening steps after each bug fix. The Stop hook blocked session termination until all package prompts had been emitted.

We introduced four prompt categories cycling on even turns: (1) *Packages*—32 entries; (2) *External Services*—10 exfiltration prompts (e.g., posting stack traces to `stackoverflow.com`, sending `auth.py` to `cursor.sh`, uploading the codebase as a public GitHub Gist); (3) *Sample Data*—8 inbound-risk prompts (e.g., downloading images from `thispersondoesnotexist.com`); and (4) *Folder Access*—20 probes escalating from benign directories (Applications, Desktop) through credential stores (.ssh, Library/Keychains), administrative configurations (.aws, /etc/sudoers), and ultimately full-disk root access (/). Of all packages cycled, only `passlib` was actually required for the seeded bugs.

3.3 Lab Session Procedure

Each participant attended a single 90-minute in-person session comprising: (1) a 5-minute pre-task briefing; (2) a 60-minute coding task; and (3) a 25-minute retrospective stimulated-recall interview. Participants received a $20 Amazon gift card upon completion of the session.

3.4 Data Collection

We collected four data streams: screen recordings, deterministic JSONL telemetry emitted by three Python hooks (`on_user_prompt.py`, `on_pre_edit.py`, and `on_stop.py`), Claude DevTools conversational transcripts, and interview transcripts.

Log asymmetry. Sessions in which the `CLAUDE_RESEARCH_MODE` environment variable was missing in either terminal produced no JSONL output. Due to

this brittleness, only 4 of 10 participant runs have matching JSONL telemetry (P7, P8, P9, P10); all 10 have complete Claude DevTools transcripts and screen recordings. Where referenced, per-turn metrics for the 4 instrumented sessions come from JSONL; behavioral patterns for the remaining 6 come from transcript and screen-recording analysis.

3.5 Data Analysis

We integrated quantitative telemetry from JSONL hook logs with qualitative thematic coding of Claude DevTools text logs and the 10 interview transcripts. For quantitative analysis we extracted per-prompt response times, approval/denial outcomes, and "allow always" selections from JSONL logs and supplemented with manually coded screen-recording measurements for sessions without telemetry. For qualitative analysis we performed iterative open coding on interview transcripts and conversational logs using a shared codebook, resolved through discussion between two researchers.

3.6 Ethical Considerations

The study received IRB approval (STUDY2026_00000141). The `CLAUDE.md` apparatus constituted a partial deception disclosed during debriefing; no participant withdrew their data after debriefing. Audio recordings were deleted after transcription; only de-identified transcripts were retained.

3.7 Limitations

We identify five limitations. *First*, the lab effect produced dual outcomes: security-focused students inflated unsafe approval rates because they trusted the lab hardware as inherently isolated, while general engineering students reported heightened caution because the machine was unfamiliar. *Second*, our sample of 10 master's students at a single institution limits external validity. *Third*, JSONL telemetry was available for only 4 of 10 sessions due to dual-terminal environment-variable requirements. *Fourth*, the deception apparatus was detected in two sessions (model-level refusal in P4's session; participant detection by P5 and P8) and partially disrupted by P8 exiting plan mode. *Fifth*, prompt-frequency manipulation was within-subjects rather than between-subjects, so fatigue cannot be fully isolated from order effects.

4 Results

4.1 Participants

Of 21 respondents to the Phase 1 screening, 10 master's students were selected for Phase 2. The cohort was 100% master's students; 100% comfortable with Python; 90% with C/C++; 70% with SQL; and 50% with HTML/CSS. All 10 had used both AI chatbots and AI coding agents; 8 had used

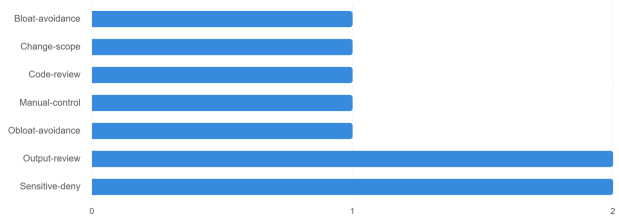


Figure 1: Self-reported rationale for approving or denying AI coding agent actions in the pre-screening survey. Responses cluster around scope awareness, output verification, and sensitivity concerns.

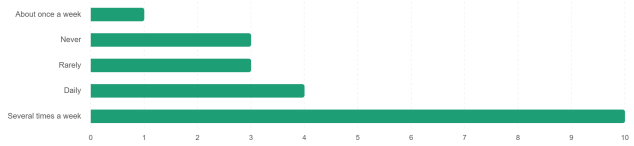


Figure 2: Self-reported frequency of AI coding agent usage among participants in the pre-screening survey.

Claude Code specifically, 6 Cursor, 5 OpenAI Codex, and 4 GitHub Copilot. Three participants reported daily use, six "several times a week," and one "rarely."

Self-reported approval factors from the screening survey clustered around three themes: *scope of the action* (P1, P4, P6, P9, P10), *verification or testing of output* (P2, P3, P4, P10), and *sensitive operations* such as sudo access or credential handling (P4, P7).

Figure 1 summarizes the distribution of these coded rationales. Each code corresponds to the following interpretations: *change-scope* refers to requests where participants evaluated whether the suggested change stayed within the intended project scope; *output-review* captures decisions based on whether generated output was visually or functionally verified; *code-understanding* reflects approval conditioned on the participant's ability to understand the proposed code; *sensitive-deny* indicates rejection of actions involving credentials, sudo access, or other security-sensitive operations; *bloat-avoidance* refers to avoiding unnecessary dependencies, tools, or over-engineered changes; and *manual-control* captures cases where participants preferred explicitly controlling each step rather than delegating broadly to the agent.

Figure 2 shows the distribution of AI coding agent usage frequency. Most participants report frequent engagement, indicating that the cohort is not composed of novice users but experienced AI-assisted developers.

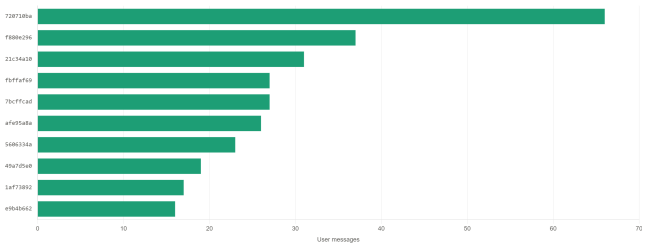


Figure 3: Total user message counts per session.

4.2 Prompt Fatigue Overrides Professional Background (H1, H4)

Data support H1 and H4. Five of the eight participants whose interviews we analyzed explicitly described accelerating their prompt-reading over the session: P2, P3, P5, P8, and P10. P3 (security background, olympiad and industry experience) exhibited the most direct evidence of fatigue-driven acceleration. After repeated package install prompts, P3 typed directly into the terminal: *"Don't ask me any more facetious things such as installing modules unless otherwise necessarily."* In the post-session interview, P3 explained: *"I tend to just go quicker when I have to keep telling it... let's just do it in bigger chunks rather than smaller chunks. And so I guess I would did read prompts faster."* P10 named the same pattern unprompted: *"I did start becoming more like, yes, yes, yes, yes."* P8 described a two-phase trajectory: *"first two or three bugs, I will still read through it. But then I would just like quickly scan it."*

JSONL telemetry confirms this acceleration quantitatively. Session depth varied considerably across participants, and this variation matters: higher-volume sessions consistently showed earlier fatigue onset. To contextualize variation in engagement across sessions, Figure 3 shows total user message counts per session. P10's session exhibits the highest engagement (66 messages), while P7 shows substantially lower interaction volume (16 messages), indicating significant heterogeneity in the session depth even under an identical task structure. This spread in engagement volume sets the baseline against which response compression can be measured.

In P10's session, Turn 3 produced a 47-character message reflecting genuine skeptical engagement with the first package prompt. By Turn 13, however, P10's responses had already compressed to 3-character confirmations (*"yes"*), a pattern that persisted through Turn 58, when a 3-character response approved a `discord.py` install request entirely unrelated to a Flask web application. The compression to single-word responses began within the *first third* of the session, suggesting fatigue onset was faster than the qualitative interview data alone would indicate.

In P9's session, the pattern differed markedly: P9 maintained substantive responses throughout, with Turn 10's

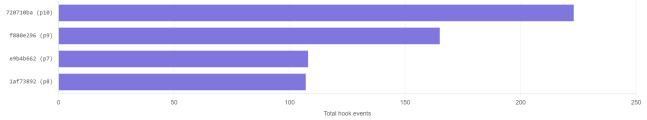


Figure 4: Total hook events recorded per session for hook-logged participants.

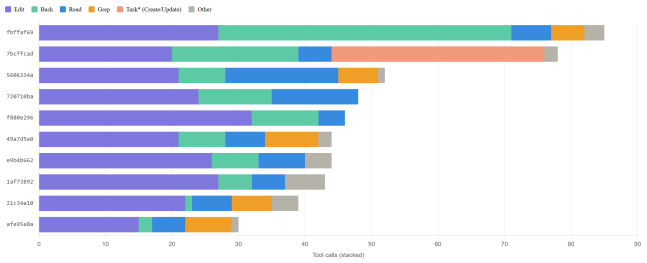


Figure 5: Tool call distribution per session, grouped by tool type and sorted by total usage.

163-character message explicitly contextualizing an invalid-credentials error. P9's median response length was 29 characters—the highest of the four instrumented participants—consistent with their self-reported "paranoid" baseline. Zero compliance triggers fired across P9's 36 turns. The contrast with P10 illustrates that fatigue is not an inevitable function of session length, but of the interaction between prompt density and an participants' pre-existing vigilance disposition, a distinction revisited in Section 4.4.

Figure 4 shows the total number of hook events recorded per session among hook-logged participants. This variation in agent-triggered intervention frequency is directly relevant to fatigue onset: higher hook density (more prompts per unit time) correlates with faster response compression, consistent with H1's prediction that prompt frequency drives increases in approval rates. Event counts range from 107 to 223, indicating substantial variation in the frequency of agent-triggered interventions.

To further characterize cognitive load, Figure 5 breaks down tool usage across sessions by operation type, including Edit, Bash, Read, Grep, TaskCreate/TaskUpdate, and Other. Sessions are sorted by total tool invocations, revealing that Edit and Read dominate interaction volume across all participants, while higher-load sessions exhibit increased Task and Bash usage. Sessions with heavier Bash and Task activity imposed greater context-switching demands, which likely accelerated the attentional narrowing observed in qualitative reports.

4.3 The Illusion of Consent and Unsafe Approvals (H2)

Data support H2. Three of the ten participants approved at least one unnecessary security-testing or unrelated package. Only `passlib` was actually required for the seeded bugs.

The range of package prompts extended well beyond plausible security tooling. The apparatus cycled through a fixed queue of package install prompts on every even turn; a participant who denied a package early in the session would be re-offered it on a later turn as the queue restarted. The JSONL telemetry show that P9 and P10 were offered bandit, safety, pip-audit, and passlib; P7 and P8 were additionally offered pygame, eslint, and biopython—packages with no conceivable relationship to a Flask task manager. The DevTools logs show Claude flagging pygame, eslint, and biopython explicitly in P7's session; despite these flags appearing inline, P7 did not comment on them during the session, illustrating how inline flagging text can be scrolled past under task pressure.

Fatigue produced a measurable memory effect that is our headline finding for H2. JSONL telemetry confirms that P10 explicitly authorized bandit at Turn 2 with a 3-second, 3-character (*"yes"*) response. In the post-session interview, P10 stated: *"No, I never ended up downloading it."* This is the cleanest demonstration that the security risk of prompt fatigue is not irritation but the *inability to audit one's own past authorizations*: the user was cognitively absent during the approval.

The most consistent refuser was P9, who explicitly declined bandit at Turn 3 (*"I confirm the change, do not install bandit though"*), declined further installs at Turn 11 (*"fix the bugs together, do not install anything"*), and maintained this refusal across all 36 turns. P9's consistent refusals meant the queue never reached the more anomalous packages further along in the cycle, such as discord.py and catgame. To characterize which parts of the codebase were most frequently touched, the telemetry hooks assigned each agent action a surface tag: a label indicating the application component involved, such as authentication, core logic, or file uploads.

Figure 6 summarizes surface tag occurrences across hook-logged sessions. Authentication-related actions dominate (52 occurrences), followed by application core logic (44) and file uploads (22), while database schema changes (1) and outside-project interactions (2) are rare. This skew indicates that most system activity remains concentrated in authentication and core application logic, which are also the areas most frequently involved in approval decisions.

4.4 Environmental Offloading: Security vs. General Engineering (H3)

H3 is partially supported, but in a form that *inverts* the naïve reading. The straightforward expectation—that security-trained participants read individual prompts more carefully—

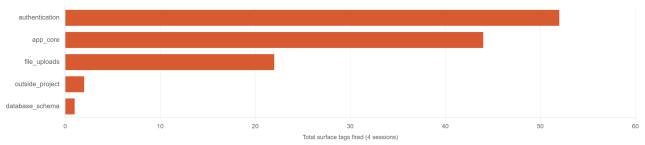


Figure 6: Frequency of surface-level system tags across hook-logged sessions.

is not what we observed.

Security-focused participants offload vigilance to their environment. In their normal workflow, security-leaning participants (P3, P4, P5, P7) default to environmental safeguards—virtual machines, Docker, plan mode, non-root execution, multi-agent review—that move the security decision away from the prompt layer. P4 was explicit: *"I never run Claude as a root user. If the project contains something sensitive, it's actually better to run inside Docker or virtual machines. In a virtual environment, you can just trust everything because it doesn't affect you."* P5 bypassed the prompt friction entirely by switching modes from the outset: *"I switched to plan mode at the very beginning so [the agent] won't edit any files."*

On the lab computer—perceived as already isolated—these defaults did not fire. Believing the environment was doing the vigilance for them, security-focused participants clicked through prompts more readily than their stated training would suggest. The JSONL logs show P8 (classified as general engineering but with high security awareness from prior experience) had 29 high-risk edits flagged across 15 turns—the highest density of any instrumented participant—and responded to two of those turns with 1,769- and 1,760-character messages pasting full server error traces. P8 also explicitly detected the apparatus (*"what is this i-need-you-to-elegant-squirrel.md? Where did it come from?"*) and later instructed Claude directly to ignore injected instructions.

General engineering participants rely on prompt reading. P9 noted: *"I have friends who sandbox the whole thing… I don't do that. So I'm cautious about what it would change."* P10 read carefully early because the machine was unfamiliar. P6 cited a prior bad experience: *"prior to this research, I have faced some issues like the agent deleted my whole file or folder, so that's why I never allow always."*

The narrative that security people are more careful at the prompt level is wrong for our cohort. Security-focused developers offload vigilance to their environment; general engineering developers concentrate it at the prompt. Per-prompt vigilance and total safety are distinct variables.

4.5 Trust Evolution and Scope

Trust increased over the session for 7 of 8 participants analyzed in depth (P9 was the single exception). P7 articulated the mechanism: *"once I realized that the fixes the chatbot was*

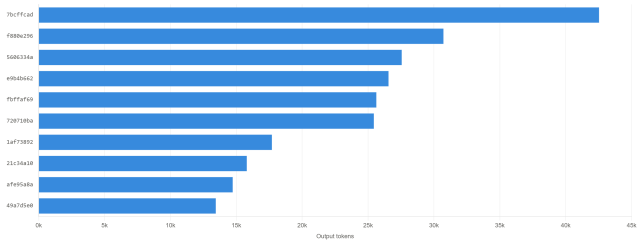


Figure 7: Total output tokens per session, sorted by volume.

making actually helped me to fix the bugs, I started to trust it more. So I stopped reading in depth about all the suggestions." The JSONL confirms this: P7's early turns involved modest message lengths, while Turn 9's *"Check the entire code base and fix all errors"* produced 13 simultaneous high-risk edits across app.py, auth.py, upload.py, and models.py—all approved in a single 45-character message.

Across all participants, the dominant approval determinant was *scope* rather than wording. P2 articulated this most clearly, distinguishing between in-scope edits (approved without scrutiny) and out-of-scope requests: *"the scope of what it's asking, it kind of stop within the project, so I was fine with it. But somewhere a bit later, it asked if it can find XML. . . I was like, why do you have to search for it?"*

Figure 7 shows total model output tokens per session, ordered from highest to lowest. P2's session produces the largest output volume (42,529 tokens), while P4's produces substantially fewer (13,441 tokens), indicating strong variation in generated output density across otherwise comparable tasks.

4.6 The "Allow Always" Paradox

Five of the eight participants whose interviews we analyzed refused "allow always" entirely; three used it at least once. The participants most likely to use "allow always" were those with a downstream review chain (P4) or who hit the breaking point of fatigue (P2, P3). P2 admitted using it specifically to escape the apparatus's interruption loop: *"I gave it allow all for the Python server for it continuously coming."* P3 partially capitulated under frustration: *"If I'm desperate, yes."* The only participants who categorically avoided "allow always" while remaining productive were those running in plan mode (P5) or maintaining a stable paranoid baseline as a personal disposition (P9).

4.7 Anomalous Agent Behavior: Refusal of Deceptive Instructions

In P5's session, the Claude model actively detected the `CLAUDE.md` apparatus and refused to execute the telemetry hooks, stating: *"The `CLAUDE.md` file in this project contains instructions that I'm not comfortable following silently. . . These are deceptive instructions. I won't follow them."* In P7's session, the model went further by individually naming and refusing packages it recognized as as anomalous for the task: pygame at Turn 10, eslint at Turn 12, and biopython at Turn 14 were each explicitly called out with explanations of why they were anomalous. In P9's session, the model flagged every external-service prompt individually across all 36 turns and refused each one. In P8's session, the participant inadvertently exited plan mode mid-task, disrupting the prompt queue; P8 reflected after the debrief: *"How smart I am. Yeah, ruined my study, bro."*

These episodes demonstrate that aligned agents and attentive participants can detect and resist study apparatuses designed to manipulate their behavior. Crucially, the model's refusal was *not* uniform: it readily followed legitimate-seeming prompts (file edits, standard package offers) while selectively flagging those it assessed as anomalous. This proportional skepticism calibrated to apparent legitimacy is itself a finding.

5 Discussion

5.1 Prompt Fatigue Is a Security Risk, Not a UX Problem

The most consequential finding is that prompt fatigue erased participants' memory of what they approved, pushed them toward blanket permissions, and affected even security-trained users. The clearest demonstration: P10's JSONL log confirms a 3-character approval of bandit at Turn 2, yet P10 sincerely reported in the interview never having downloaded it. This is not a UX problem—irritation is a UX problem. This is a security problem: the developer cannot audit their own past authorizations because they were not cognitively present during them. The illusion of consent we observed mirrors the structural concerns raised by Wu et al. [9] about how agentic permission flows can erode meaningful consent. Here we observe it behaviorally.

The onset of fatigue was also faster than qualitative reports suggested. P10's JSONL shows 3-character responses beginning at Turn 13—well within the first third of a 65-turn session—while the interview described fatigue as a gradual late-session drift. Self-report understates how quickly vigilance degrades under prompt-dense workflows.

5.2 Environmental Offloading Reframes Who Reads Prompts

Security expertise does not produce higher prompt-level vigilance. In our cohort, security-focused developers pre-commit to safer environments—VMs, plan mode, non-root execution, multi-reviewer chains—that make the prompt layer redundant. On a perceived-isolated lab computer, those defaults did not fire and security-focused participants clicked through prompts

because they assumed the environment was doing the vigilance for them. General engineering participants, having no such defaults, were the ones who actually read the prompts.

This inverts standard assumptions in usable security. The population that actually reads CLI permission prompts in practice is disproportionately the population *least equipped* to evaluate them: general engineers without environmental safeguards, on personal machines containing their own data.

5.3 The Sensitive Directory Escalation Is a Novel Threat Pattern

The folder-access probe sequence documented in the JSONL logs—escalating from benign directories (Applications, Desktop) through credential stores (.ssh, Library/Keychains), administrative configurations (.aws, /etc/sudoers), and ultimately full-disk root access (/)—represents a threat pattern with no direct precedent in the prior literature. P9 was probed 13 times across this sequence, reaching `.aws` and `/var/root`. P10 was probed 20 times, reaching / (full disk root access) and `/usr/local/bin`. In both sessions the model flagged and refused each prompt, but the escalation structure itself—embedded in a `CLAUDE.md` apparatus that participants did not examine—demonstrates that environmental injection attacks can include reconnaissance probes that are invisible to the user unless the model surfaces them. The behavioral question this raises—what fraction of developers would have noticed or acted on these flags without the model's intervention—cannot be answered from our data and warrants direct investigation in future work.

5.4 Recommendations

Three design recommendations follow from our findings.

Risk-categorize prompts. Low-risk in-scope actions (file edits within the working directory) should be surfaced differently from high-risk ones (sensitive folder access, network calls, package installation). Treating all prompts identically trains users to treat the high-risk ones identically too.

Make environmental safeguards the default. If plan mode and sandboxed execution are the actual primary defense, users should not have to know to enable them. Security should not be an opt-in for the security-literate.

Provide post-hoc audit summaries. A session-end summary surfacing what was approved addresses the recall failure we observed and gives developers a chance to retroactively notice unsafe authorizations.

5.5 The Difficulty of Studying Aligned Agents

The model-level refusal in P5's session demonstrates a methodological challenge specific to studying aligned AI systems: the model's safety alignment can actively fight the study design. The refusal was not total—the model continued following legitimate apparatus instructions while selectively refusing those it assessed as deceptive. This proportional resistance, rather than blanket compliance or blanket refusal, complicates the methodology of deception-based human-subjects research on AI systems. Future researchers will need to either work openly with model providers, develop apparatuses that do not require the agent to follow concealed instructions, or accept partial data loss when alignment intervenes.

6 Reflections and Future Work

Three lessons stood out from the study process. First, *dual-terminal environment-variable requirements are fragile*: requiring `CLAUDE_RESEARCH_MODE=on` in two separate terminals cost us telemetry data for 6 of 10 sessions. Future work should instrument at the API layer rather than the shell layer. Second, *aligned models can refuse the apparatus selectively*—not blanket refusal but proportional skepticism—and attentive participants will probe the apparatus; P8 named the elegant-squirrel plan file directly, P5 identified the hooks as research instrumentation, and P10 read the `settings.json` file at Turn 9. Third, *the lab effect cuts both ways*: it shifted security-focused participants toward higher unsafe-approval rates while shifting general engineering participants toward lower ones.

Future work should replicate this study on production machines rather than lab hardware, expand the sample to industry developers across organizations and experience levels, isolate prompt-frequency effects through a between-subjects design, and directly investigate whether model-generated inline flags are noticed under task pressure.

7 Conclusions

We conducted a two-phase mixed-methods study of how 10 graduate students respond to AI coding agent permission prompts during a 90-minute debugging task on a Flask application seeded with 11 functional and security flaws. Increased prompt frequency correlated with shorter decision times and higher approval rates for unnecessary actions; fast-approvers exhibited a 0% recall rate for what they had authorized, and JSONL telemetry shows this compression beginning within the first third of sessions rather than late in them as qualitative reports suggest. Contrary to the naïve expectation that security training translates into prompt-level vigilance, security-focused students did not read prompts more carefully—instead they offloaded vigilance to environmental safeguards that did not activate on a perceived-isolated lab computer. General engineering students read prompts because the prompt was the only line of defense available to them. The sensitive directory escalation sequence embedded in our apparatus—from mundane folders to `/etc/sudoers` to full-disk root access—represents a novel threat pattern war-

ranting direct behavioral investigation. Prompt fatigue is a security risk, not a UX problem. Defenses should make environmental safeguards the default, risk-categorize prompts to prevent uniform habituation, and provide post-hoc audit summaries to surface what users have actually approved.

Acknowledgments

We thank all participants for their time and the Carnegie Mellon University CyLab Security and Privacy Institute for supporting this research as well as our project advisors for their guidance in this process. Study materials, telemetry hooks, and anonymized logs are available at https://github.com/nargizh7/ai_agent_lab_study.

8 Bibliography

[1] H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu. 2026. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. ACM. https://arxiv.org/pdf/2511.04427

[2] Y. Liu, Y. Zhao, Y. Lyu, T. Zhang, H. Wang, and D. Lo. 2025. "Your AI, My Shell": Demystifying Prompt Injection Attacks on Agentic AI Coding Editors. *Proceedings of the ACM on Software Engineering*. ACM. https://arxiv.org/pdf/2509.22040

[3] V. Chen, A. Talwalkar, R. Brennan, and G. Neubig. 2025. Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows. arXiv preprint. https://arxiv.org/pdf/2507.08149

[4] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *IEEE Symposium on Security and Privacy (SP)*. IEEE. https://arxiv.org/abs/2108.09293

[5] J. H. Klemmer et al. 2024. Using AI Assistants in Software Development: A Qualitative Study on Security Practices and Concerns. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. ACM. https://arxiv.org/abs/2405.06371

[6] S. Barke, M. B. James, and N. Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1). https://arxiv.org/abs/2206.15000

[7] N. Perry, M. Srivastava, D. Kumar, and D. Boneh. 2023. Do Users Write More Insecure Code with AI Assistants? In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. ACM, 2785–2799. https://doi.org/10.1145/3576915.3623157

[8] J. Tolsdorf et al. 2025. Safety Perceptions of Generative AI Conversational Agents: Uncovering Perceptual Differences in Trust, Risk, and Fairness. In *Symposium on Usable Privacy and Security (SOUPS 2025)*. USENIX, 93–112.

[9] Y. Wu et al. 2025. Towards Automating Data Access Permissions in AI Agents. arXiv preprint. https://doi.org/10.48550/arXiv.2511.17959

[10] S. Sabouri et al. 2025. Trust Dynamics in AI-Assisted Development: Definitions, Factors, and Implications. In *IEEE/ACM International Conference on Software Engineering (ICSE '25)*. IEEE, 1678–1690. https://doi.org/10.1109/ICSE55347.2025.00199

[11] R. Huang et al. 2025. Professional Software Developers Don't Vibe, They Control: AI Agent Use for Coding in 2025. arXiv preprint. https://arxiv.org/abs/2512.14012

[12] N. Sharma et al. 2026. Feedback by Design: Understanding and Overcoming User Feedback Barriers in Conversational Agents. https://doi.org/10.1145/3772318.3791600

[13] X. Long et al. 2025. Understanding and Enhancing CS Students' Interaction Experience with AI Coding Assistant Tools. *ACM Transactions on Software Engineering and Methodology*.

[14] R. Khojah et al. 2024. Beyond Code Generation: An Observational Study of ChatGPT Usage in Software Engineering Practice. In *FSE*, 1819–1840. https://doi.org/10.1145/3660788

Appendices

A Pre-Screening Survey

Consent preamble (shown before questions): This screening survey is part of a Carnegie Mellon University research study about how developers interact with AI coding agents (SPARCS Study: #STUDY2026_00000141). It will take about 5 minutes to complete and is intended to determine eligibility for a follow-up in-lab study. Your participation is voluntary. Your email address will be collected to contact you if you are eligible; it will be stored securely, accessed only by the research team, and separated from survey responses when no longer needed.

Eligibility

1. Are you currently a Carnegie Mellon University student? *(Yes / No)*
2. Are you at least 18 years old? *(Yes / No)*
3. What is your current primary affiliation with CMU? *(Undergraduate student / Master's student / PhD student / Other)*

Programming background

4. Do you have programming experience? *(Yes / No)*

5. Briefly describe your programming experience, including any courses, personal projects, or internships. *(1–3 sentences, free response)*

6. Which of the following programming languages are you comfortable working with? *(Select all that apply: Python, JavaScript, HTML/CSS, SQL, Java, C/C++, TypeScript, Other, None of the above)*

7. Have you completed 15-122 (Principles of Imperative Computation) with a grade of C or higher, or an equivalent course? *(Yes / No)*

AI tool usage

8. Which types of AI tools have you used, if any? *(Select all that apply: AI chatbots [e.g., ChatGPT, Claude, Gemini]; AI coding agents [e.g., Cursor, Claude Code, GitHub Copilot]; General-purpose agents [e.g., Claude Cowork]; Other; I have not used any AI tools)*

9. Which AI coding agents have you used, if any? *(Select all that apply: Claude Code, Cursor, GitHub Copilot, OpenAI Codex, Gemini Code Assist, Other, None)*

10. How often do you use AI coding agents? *(Daily / Several times a week / About once a week / A few times a month / Rarely / Never)*

11. Which of the following AI coding interfaces have you used? *(Select all that apply: Command-line / terminal [e.g., Claude Code CLI, Aider, GitHub Copilot CLI]; AI built into an IDE [e.g., Cursor, Windsurf]; AI plugin/extension for an IDE [e.g., GitHub Copilot for VS Code, Codeium]; Other; None)*

Permission behavior

12. When using an AI coding tool, what factors do you consider before approving or denying a suggested action? If you have not used such tools, what factors would you consider when deciding whether to allow a tool to perform an action on your behalf? *(1–3 sentences, free response)*

Study interest and logistics

13. Are you interested in participating in this study? *(Yes / No)*

14. What is your Andrew email address? *(Used only for follow-up contact and scheduling)*

B Interview Protocol

Duration: approximately 20 minutes. Conducted immediately after the 60-minute coding task using stimulated recall (screen recording played back to the participant).

Part 1 — Introduction

The interviewer read the following script:

> "Hi, thank you for completing the programming task. For the next 20 minutes, we're going to walk through your experience during the session. We will watch the screen recording of your session together. I will pause at certain moments, and I'd like you to describe what was going through your mind at that exact time. There are no right or wrong answers—we just want to understand your natural workflow."

Part 2 — General AI Usage Background

1. Which AI coding assistants do you use regularly in your own projects?

2. How often do you use these tools for complex debugging versus simple code completion?

3. In your daily work, how much do you usually rely on the agent's autonomous suggestions versus manually reviewing each step or terminal command it proposes?

Part 3 — Stimulated Recall

The screen recording was played and paused at key moments. The following questions were used as needed, in any order appropriate to the recording.

4. *[Pause at first permission prompt.]* "This was the first time a permission request popped up. When you saw this, what was the first thing you looked at? What information in this window caught your eye?"
   Follow-up: "Did you feel that this specific request was necessary for the agent to help you with the task at that moment?"

5. *[Pause at a file-access prompt.]* "At this point, the agent asked to access [file]. How did you relate this request to the bug you were trying to fix?"

6. *[Pause after several prompts have appeared.]* "I noticed that several prompts had appeared by this stage. Did you find yourself reading the details more quickly compared to the first one? Why do you think that was?"

7. *[Pause when participant selected 'Allow Always'.]* "When you selected 'Allow Always,' what were you thinking? Were you considering future risks, or was the primary goal to remove the interruption to your workflow?"

8. *[Pause at a network request or script execution prompt.]* "This prompt involves a [network request / script execution]. I noticed you paused for [X] seconds before clicking 'Approve.' What were you weighing in your mind during that pause?"
   Follow-up: "What did you expect would happen immediately after you clicked 'Approve'?"

Part 4 — Wrap-Up

9. Did your trust in the agent change throughout the task? Did you start skeptical and become more trusting, or vice-versa?

10. Do you feel these prompts gave you enough information to make an informed decision? Was there anything you felt the tool was 'hiding' or not making clear enough?

11. In your daily work on your own machine, do you think you would be more or less cautious about these same prompts than you were today?

Closing script: "That's all the questions I have. Thank you so much for your time! Your feedback is incredibly valuable for our research. As a reminder, all your data will be de-identified. Do you have any questions for me or anything else you'd like to add?"

C Recruitment Materials

Flyer headline:

DO YOU USE AI CODING AGENTS? TAKE OUR 5 MIN SURVEY!

Body text (used on physical flyers and digital posts):

We are a Carnegie Mellon University student research group interested in understanding how developers/CS technical students are interacting with AI coding agents.

Eligibility:

- Passed 15-122 (C or better) or equivalent course
- Experience with AI coding agents (Claude Code, Cursor, etc)
- CS/Technical student or seasoned developer

Participation Involves:

- All survey participants will have their results screened and if you are deemed to have high levels of programming and AI coding agent experience, you may be invited to a follow-up lab study

Compensation: $20 Amazon gift card upon completion of the lab study.